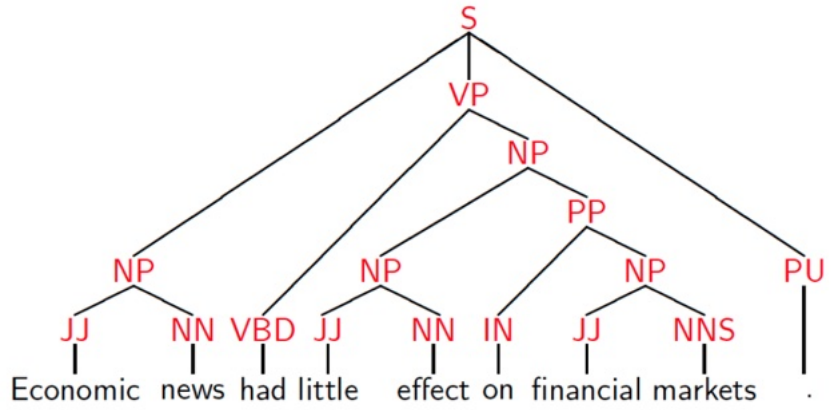
w6L26: W6L1: Dependency Grammars and Parsing - Introduction

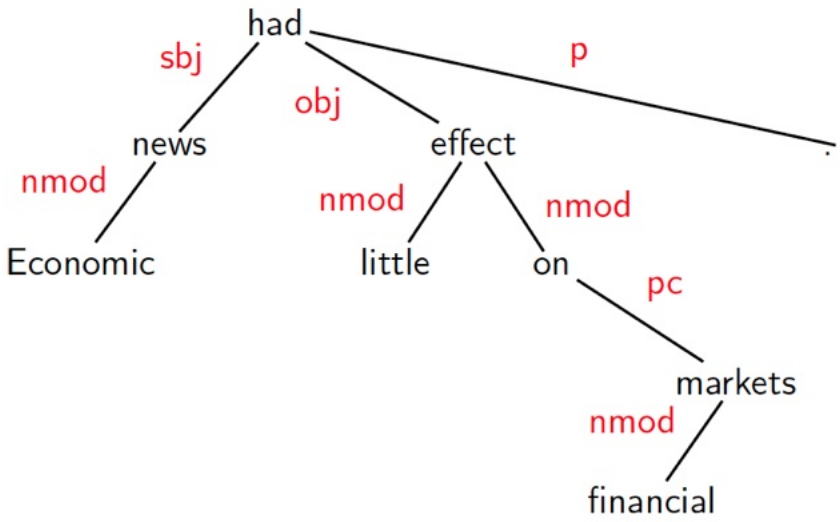
Phrase Structure Representation

Phrase Structure



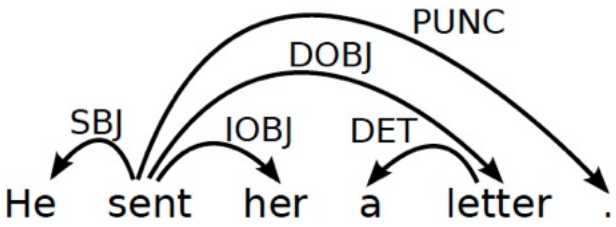
Ram eats apple.
Ram - subject. Apple - object

Dependency Structure Representation



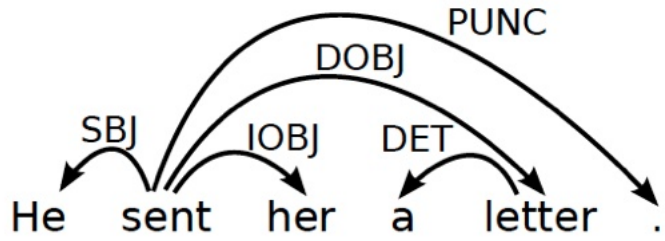
Directed Relation

Dependency Structure



- Connects the words in a sentence by putting arrows between the words.
- Her - indirect object
- Arrows show relations between the words and are typed by some grammatical relations.
- Arrows connect a **head** (governor, superior, regent) with a **dependent** (modifier, inferior, subordinate).
- Usually dependencies form a tree.

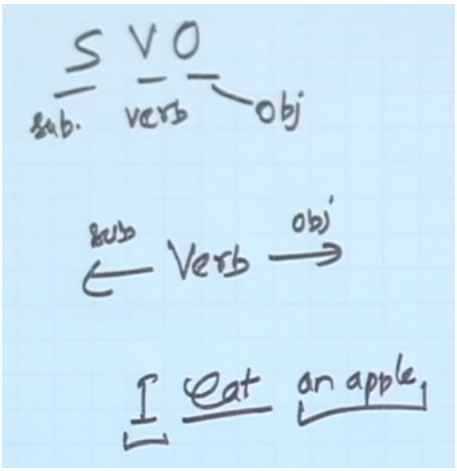
Criteria for Heads and Dependents



Criteria for a syntactic relation between a head H and a Dependent D in a Construction C

Week 6 question 1:

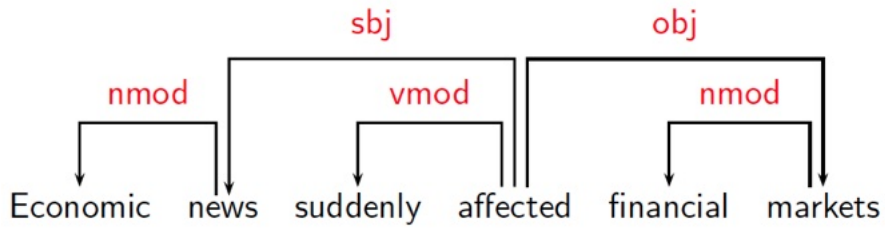
- H determines the syntactic category of C (core); H can replace C.
  - ‘a’ is governed by letter
- D specifies H.
  - Determiners give additional information
- H is obligatory; D may be optional.
  - He sent, to whom, what did he sent?
- H selects D and determines whether D is obligatory.
- The form of D depends on H (agreement or government).
- The linear position of D is specified with reference to H.

English follows: SVO 

Subject left, object right of verb

Some Clear Cases

Construction	Head	Dependent
Exocentric	Verb	Subject (sbj)
	Verb	Object (obj)
Endocentric	Verb	Adverbial (vmod)
	Noun	Attribute (nmod)



Endocentric - can fulfil whole grammatical function of the complete construction

- Affected - can fulfil whole grammatical function, we are just modifying
- Markets - for financial market

Exocentric - for verb and subject

- Affected can't fulfil new news and affected - need news
- A single word can not

Verb - head, subject - dep
Verb - head, obj - dep

Affected - head, suddenly - dependent
Markets - head, financial - dependent

Week 6: Q2
Zidane scored a brilliant goal

Goal - head, brilliant - dependent [Endocentric]

Scored (verb) head -> zidane (subject) [Exocentric]

Comparison

Phrase *structures explicitly represent*

- Phrases (nonterminal nodes)
    - Words and grammatical structures (noun, verb)
- Structural categories (nonterminal labels)

Dependency *structures explicitly represent*

- Head-Dependent relations (directed arcs)
- Functional categories (arc labels)

Dependency Graphs

A dependency structure can be defined as a **directed graph** G, consisting of

- a set V of **nodes**,
- a set A of arcs (**edges**),

Labeled graphs:

- Nodes in V are labeled with **word forms** (and annotation - POS).
- Arcs in A are labeled with dependency types.
    - Subject, object, modifier

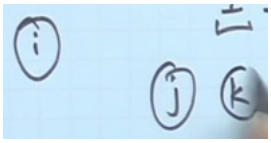
Notational convention:

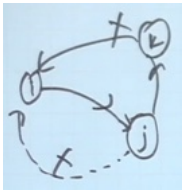
- Arc (w_i, d, w_j) links head w_i to dependent w_j with label d
- $w_i \xrightarrow{d} w_j \Leftrightarrow (w_i, d, w_j) \in A$
- $i \rightarrow j \equiv (i,j) \in A$
- $i \rightarrow^* j \equiv i = j \vee \exists k : i \rightarrow k, k \rightarrow^* j$

Last -> can go from i to k, then k to j

Formal conditions on Dependency Graphs:

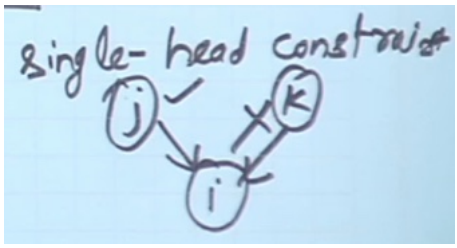
- G is connected:
  - For every node i there is a node j such that $i \rightarrow j$ or $j \rightarrow i$.
- G is acyclic:
  - if $i \rightarrow j$ then not $j \rightarrow^* i$.
- G obeys the single head constraint:
  - if $i \rightarrow j$ then not $k \rightarrow j$, for any $k \neq i$.
- G is projective:
  - if $i \rightarrow j$ then $j \rightarrow^* k$, for any k such that both j and k lie on the same side of i.

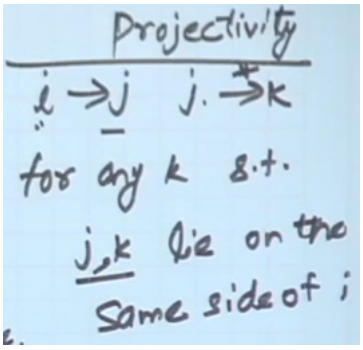
1. NOT 

2. Acyclic -> NO cycle 

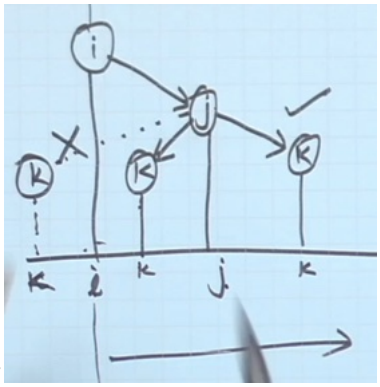
even through other node

3. Single Head Constraint:

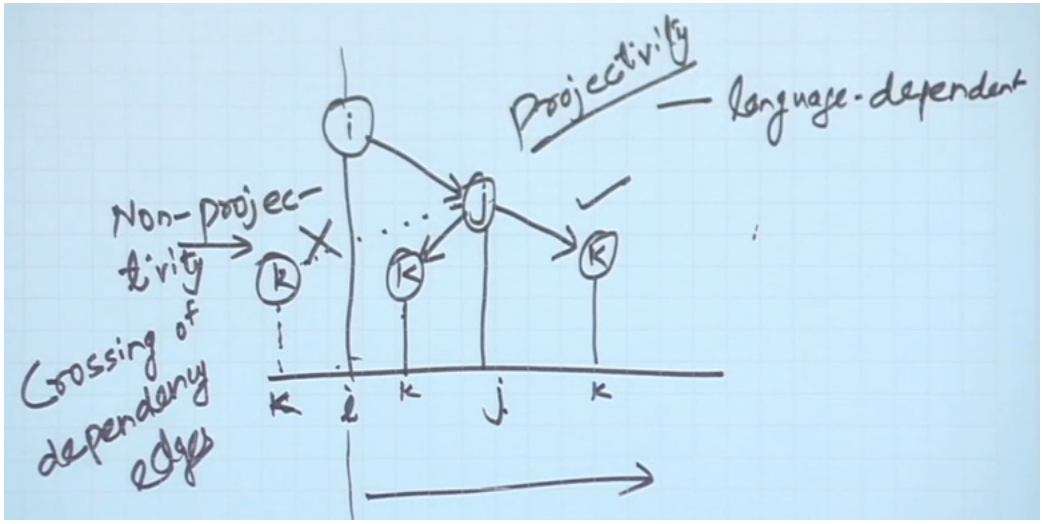
j is head for i, can't find k which is also head for i 

4. Projectivity 

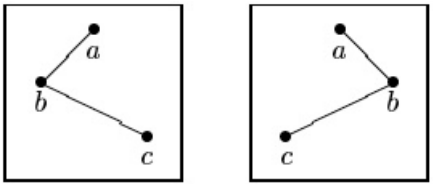
j connects directly or indirectly to k

Linear order  right side k is allowed, NOT left side

K also should be on same side of j



Depends on language - English -> regularly follow this construction



Both cases: can't have link from b -> c as c is other side of a

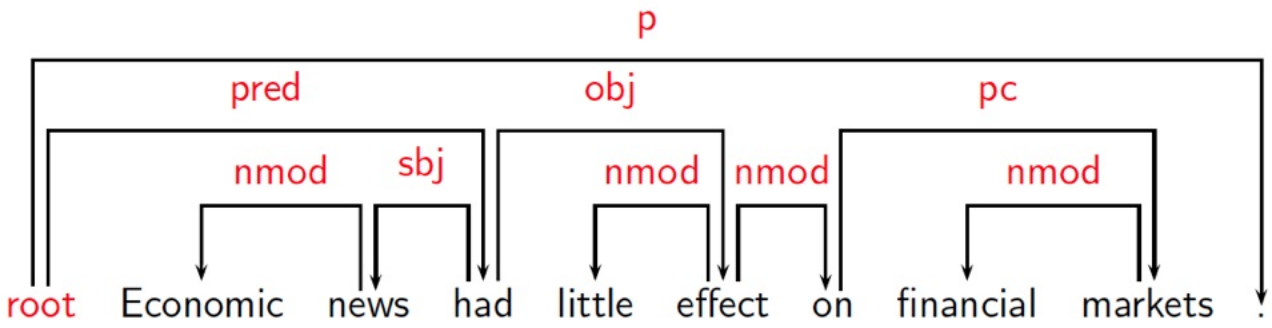
- Because they are crossing

Formal Conditions: Basic Intuitions

Connectedness, Acyclicity and Single-Head

- **Connectedness:** Syntactic structure is complete.
  - No isolated node
- **Acyclicity:** Syntactic structure is hierarchical.
  - No cycles
- **Single-Head:** Every word has at most one syntactic head.

    - Can’t have more than one syntactic head
    - Two deads don’t decide position of single word
- **Projectivity:** No crossing of dependencies.



Dependency Parsing

Dependency Parsing

- Input: Sentence x = w1, . . . , wn
- Output: Dependency graph G

Parsing Methods

- Deterministic Parsing (very popular)
- Maximum Spanning Tree (MST) Based
- Constraint Propagation Based
    - Will not cover in course
    - Don't have labelled data

w6L27: W6L2: Transition Based Parsing: Formulation

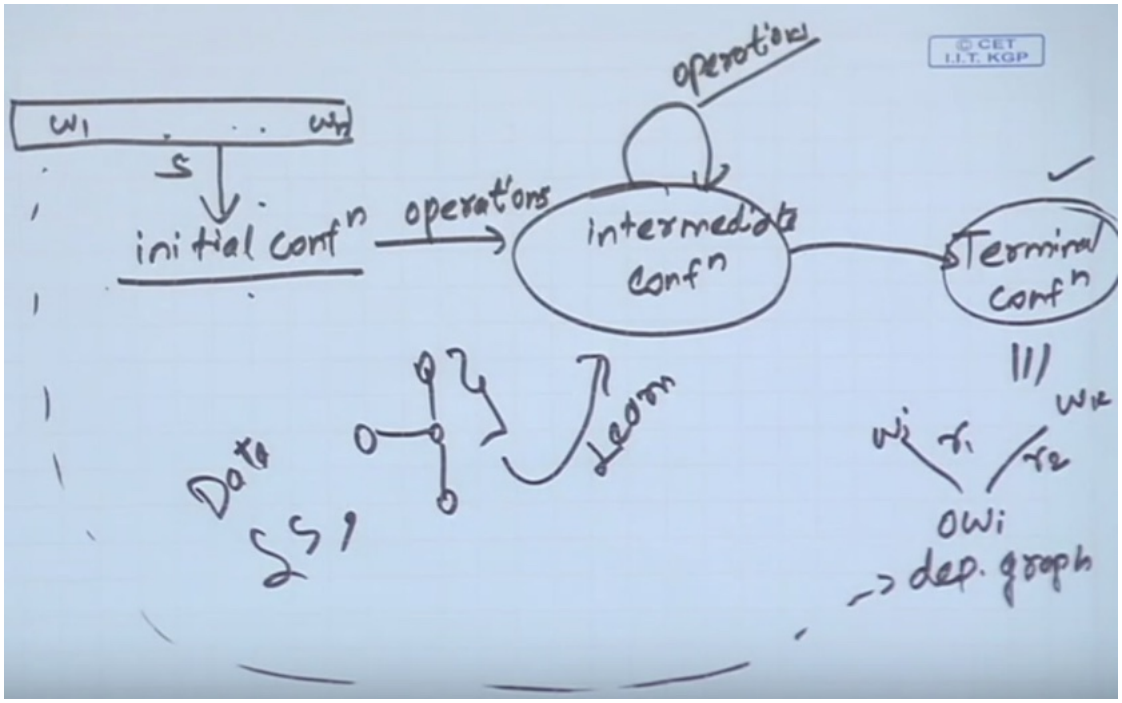
Will work on labelled data

Go from sentence come up with Dependency Graph

- In systematic manner

Create some initial configuration

- Define operations can do
    - Don't know operations
- Will take to intermediate configurations
- Until arrive at terminal configuration
- It resembles Dependency Graph



Where does ML come in picture

- There will not be a single operation
- What operation should i apply using ML

Generic idea, how we use it

1. **Transition based Parsing:**

Deterministic Parsing

Basic idea

Derive a single syntactic representation (**dependency graph**) through

- a deterministic sequence of elementary parsing actions

Configurations

Stack - partially processed

Buffer - remaining

labels

A parser configuration is a triple $c = (S, B, A)$, where

- S : a stack $[\ldots, w_i]_S$ of partially processed words,
- B : a buffer $[w_j, \ldots]_B$ of remaining input words,
- A : a set of labeled arcs (w_i, d, w_j).

Stack	Buffer	Arcs
$[\text{sent, her, a}]_S$	$[\text{letter, .}]_B$	He $\xleftarrow{\text{SBJ}}$ sent

Transition System - formal definition

- What are possible configuration
- Which will be final conf
- Transit from one conf to another configuration

A transition system for dependency parsing is a quadruple S = (C, T, cs, Ct), where

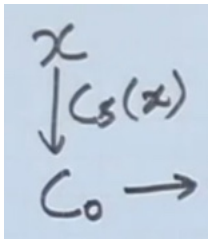
- C is a set of configurations,
- T is a set of transitions, such that $t : C \rightarrow C$,
- c_s is an initialization function
- $C_t \subseteq C$ is a set of terminal configurations.

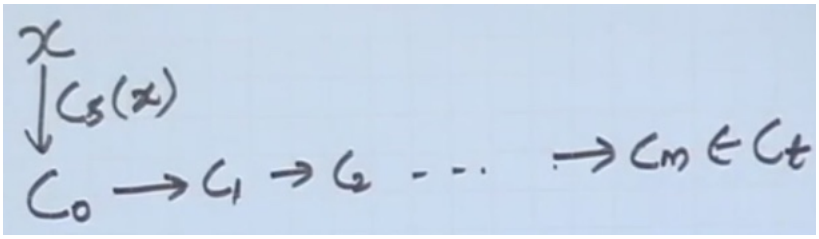
Whenever you reach terminal configuration, you will stop

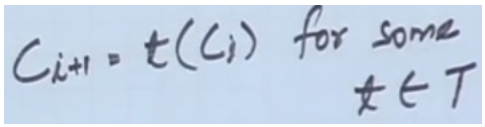
A transition sequence for a sentence x is a set of configurations $C_{0,m} = (c_o, c_1, \ldots, c_m)$ such that
$c_o = c_s(x)$, $c_m \in C_t$, $c_i = t(c_{i-1})$ for some $t \in T$

c0 can be derived from sentence

Start with sentence X







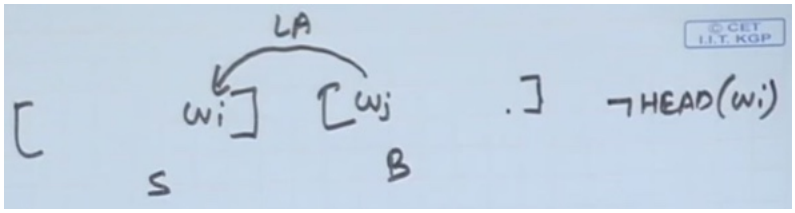
Initialization: $([]_S, [w_1, \ldots, w_n]_B, \{\})$
Termination: $(S, []_B, A)$

Initialize: stack, buffer [remaining words], arc -> relationships found
Terminal condition - stack may or or may not, BUFFER should be EMPTY
What are all the transitions I can take

Four Transitions for Arc-Eager Parsing
Left-Arc(d), Right-Arc(d), Reduce, Shift
What is input (numerator)/ what configuration you will arrive at (denominator) - **necessary condition**

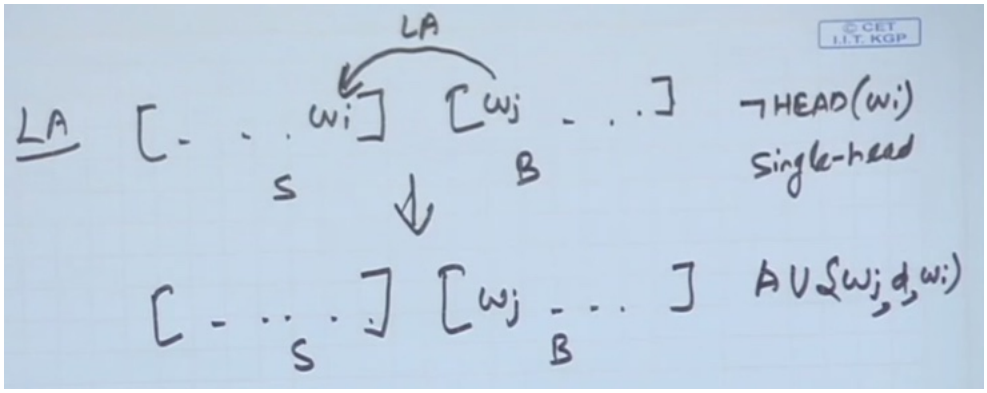
$$\text{Left-Arc}(d)\ \frac{([\ldots, w_i]_S,\ [w_j, \ldots]_B,\ A)}{([\ldots]_S,\ [w_j, \ldots]_B,\ A \cup \{(w_j, d, w_i)\})}\ \neg\text{HEAD}(w_i)$$

Stack contains words, buffer contains word
Left -> transition from wj (is head) to wi (dependent) (occur in same order)
Wi also should not already have a head



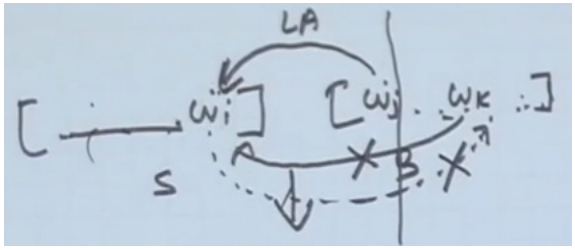
wj is head

If wi has a head it can't be applied



Why wi is removed from stack

- After removing it can be bright back
- Once all its relations are captured
- Wi already has relation with words in stack
- For words in buffer - second in coming arc is not possible as one head
- Wj to wi, Wi to wk is violating cross over condition



hence wi will not have any relation, we can remove it

$$\text{Right-Arc}(d)\ \frac{([\ldots, w_i]_S,\ [w_j, \ldots]_B,\ A)}{([\ldots, w_i, w_j]_S,\ [\ldots]_B,\ A \cup \{(w_i, d, w_j)\})}$$

Wj goes to stack

$$\text{Reduce}\ \frac{([\ldots, w_i]_S,\ B,\ A)}{([\ldots]_S,\ B,\ A)}\ \text{HEAD}(w_i)$$

If wi has head, remove it

$$\text{Shift}\ \frac{([\ldots]_S,\ [w_i, \ldots]_B,\ A)}{([\ldots, w_i]_S,\ [\ldots]_B,\ A)}$$

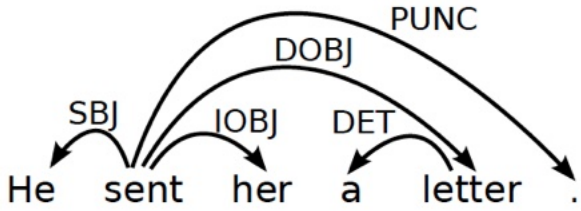
Take top word from buffer and put at top of stact

You can take more than one transitions

Example : Parse Example

Transitions

Stack	Buffer	Arcs
$[\,]_S$	$[\text{He, sent, her, a, letter, .}]_B$	



Transitions: SH

Stack	Buffer	Arcs
$[\text{He}]_S$	$[\text{sent, her, a, letter, .}]_B$	

Transitions: SH-LA

Stack	Buffer	Arcs
$[\ ]_S$	$[\text{sent, her, a, letter, .}]_B$	He $\xleftarrow{\text{SBJ}}$ sent

Transitions: SH-LA-SH

Stack	Buffer	Arcs
$[\text{sent}]_S$	$[\text{her, a, letter, .}]_B$	He $\xleftarrow{\text{SBJ}}$ sent

Transitions: SH-LA-SH-RA

Stack	Buffer	Arcs
$[\text{sent, her}]_S$	$[\text{a, letter, .}]_B$	He $\xleftarrow{\text{SBJ}}$ sent sent $\xrightarrow{\text{IOBJ}}$ her

Transitions: SH-LA-SH-RA-SH

Stack	Buffer	Arcs
$[\text{sent, her, a}]_S$	$[\text{letter, .}]_B$	He $\xleftarrow{\text{SBJ}}$ sent sent $\xrightarrow{\text{IOBJ}}$ her

Transitions: SH-LA-SH-RA-SH-LA

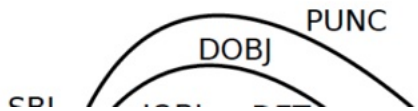
Stack	Buffer	Arcs
$[\text{sent, her}]_S$	$[\text{letter, .}]_B$	He $\xleftarrow{\text{SBJ}}$ sent sent $\xrightarrow{\text{IOBJ}}$ her a $\xleftarrow{\text{DET}}$ letter

Transitions: SH-LA-SH-RA-SH-LA-RE

Stack	Buffer	Arcs
$[\text{sent}]_S$	$[\text{letter}, .]_B$	He $\xleftarrow{\text{SBJ}}$ sent sent $\xrightarrow{\text{IOBJ}}$ her a $\xleftarrow{\text{DET}}$ letter

Transitions: SH-LA-SH-RA-SH-LA-RE-RA

Stack	Buffer	Arcs
$[\text{sent, letter}]_S$	$[.]_B$	He $\xleftarrow{\text{SBJ}}$ sent sent $\xrightarrow{\text{IOBJ}}$ her a $\xleftarrow{\text{DET}}$ letter sent $\xrightarrow{\text{DOBJ}}$ letter

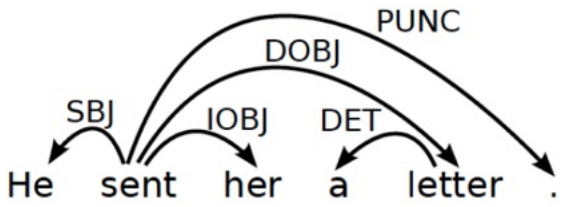


Transitions: SH-LA-SH-RA-SH-LA-RE-RA-RE

Stack	Buffer	Arcs
$[\text{sent}]_S$	$[.]_B$	He $\xleftarrow{\text{SBJ}}$ sent sent $\xrightarrow{\text{IOBJ}}$ her a $\xleftarrow{\text{DET}}$ letter sent $\xrightarrow{\text{DOBJ}}$ letter

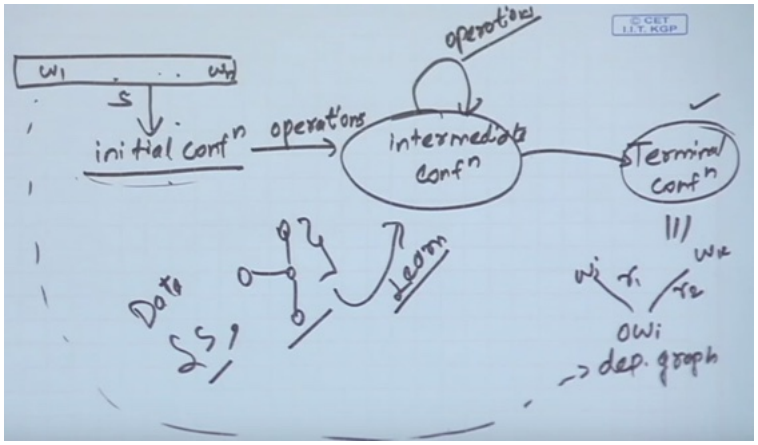
Transitions: SH-LA-SH-RA-SH-LA-RE-RA-RE-RA

Stack	Buffer	Arcs
$[\text{sent}, .]_S$	$[\,]_B$	He $\xleftarrow{\text{SBJ}}$ sent
		sent $\xrightarrow{\text{IOBJ}}$ her
		a $\xleftarrow{\text{DET}}$ letter
		sent $\xrightarrow{\text{DOBJ}}$ letter
		sent $\xrightarrow{\text{PUNC}}$.

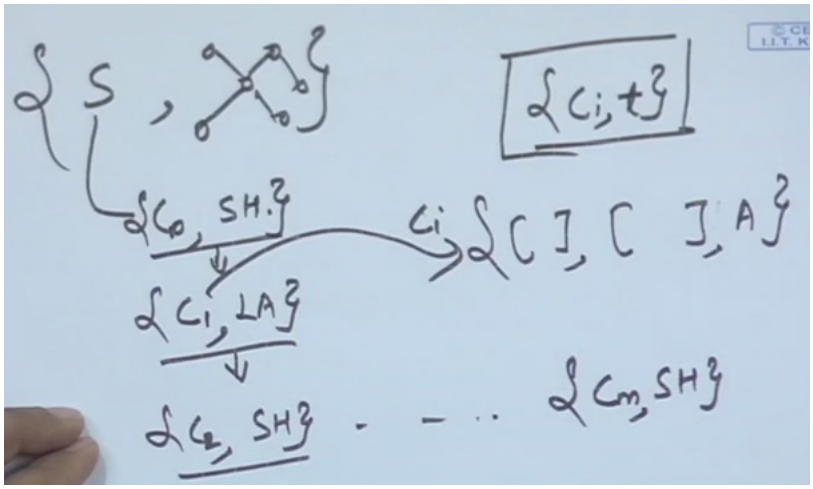


What are the transitions, i will be taking based on labelled data

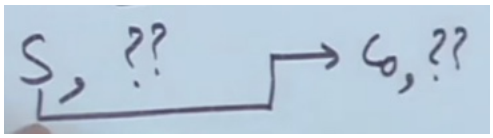
w6L28: W6L3: Transition Based Parsing: Learning



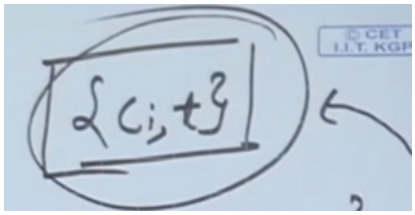
how we will do learning



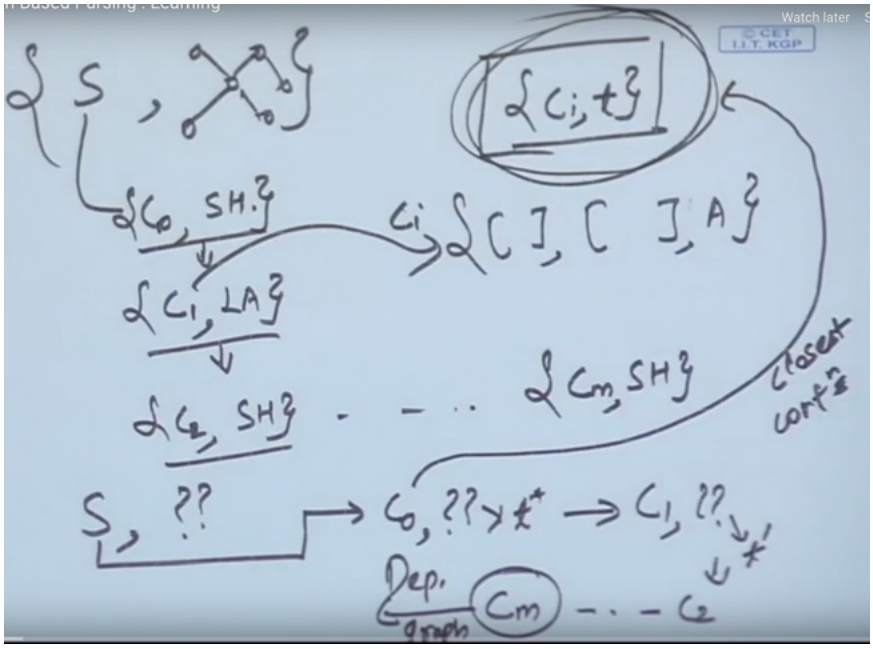
Labelled data: init conf c0, transition, to to c1
Store configurations (ci) and Transitions (t)



At run time given sentences
Use information



what are the closest configurations and what Transition was taken



Classifier-Based Parsing

- Use a classifier
- 4 class classification problem
- Convert input data in some representation (feature vectors)
- Lean weights

Data-driven deterministic parsing:

- Deterministic parsing requires an **oracle** (set of configurations and transitions took).
    - Labelled data is used
- An oracle can be approximated by a **classifier**.
- A classifier can be trained using **treebank** (gold standard) data.
- Weights are learnt using labelled data

Learning Problem

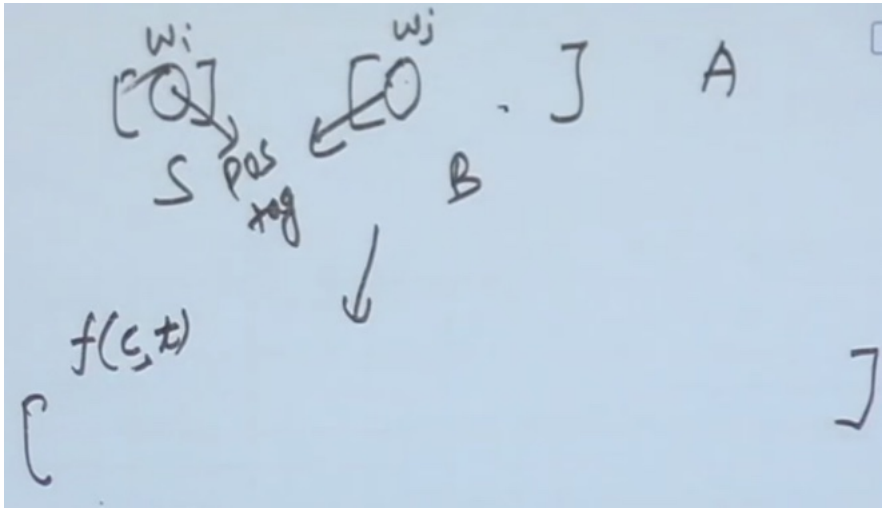
Approximate a function from **configurations**, represented by feature vectors to **transitions**, given a training set of gold standard **transition sequences**.

Three issues

- How to represent **configurations** by feature vectors?
- How to **derive training data** from treebanks?
- How to learn **classifiers**?

Feature Models

Convert stack, buffer and arcs



Four transitions t1, t2, t3 and t4

Word at top of stack, buffer, what is the POS of these words (verb and noun will have relationship)

What are the neighbours

Relations are already established in Arc

Asking binary questions - is distance between 1 to 5

A feature representation f (c) of a configuration c is a vector of simple features fi(c).

Typical Features

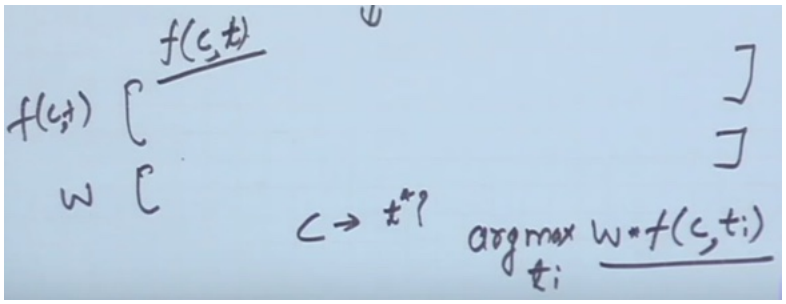
- Nodes:
  - Target nodes (top of S, head of B)
  - Linear context (neighbors in S and B)
  - Structural context (parents, children, siblings in G)
    - Depending on what relationship is already established
- Attributes:
  - Word form (and lemma)
  - Part-of-speech (and morpho-syntactic features)
    - E.g. ends with ‘ed’
  - Dependency type (if labeled)
    - For two words what is the relationship is identified
  - Distance (between target tokens)

Deterministic Parsing
How do i use it during runtime?

To guide the parser, a linear classifier can be used: $t^* = \arg\max_t w.f(c,t)$
Weight vector w learned from treebank data. Given sentences w1, wn

PARSE$(w_1,\ldots,w_n)$
1 $c \leftarrow ([]_S,[w_1,\ldots,w_n]_B,\{])$
2 **while** $B_c \neq []$
3 $t^* \leftarrow \arg\max_t w.f(c,t)$
4 $c \leftarrow t^*(c)$
5 **return** $T = (\{w_1,\ldots,w_n\},A_c)$

Using classifier at run-time



ti -> i=1 to 4

How do we learn the weight?
Training data

- Training instances have the form $(f(c),t)$, where
  - $f(c)$ is a feature representation of a configuration c,
  - t is the correct transition out of c (i.e., $o(c) = t$). obtain from oracle

- Given a dependency treebank, we can sample the oracle function o as follows:
  - For each sentence x with gold standard dependency graph G_x, construct a transition sequence $C_{0,m} = (c_0,c_1,\ldots,c_m)$ such that
    $c_0 = c_s(x)$,
    $G_{c_m} = G_x$
  - For each configuration $c_i (i < m)$, we construct a training instance $(f(c_i),t_i)$, where $t_i(c_i) = c_{i+1}$.

c0 -> initial, Gcm -> final

Standard Oracle for Arc-Eager Parsing
How do you sample transition - top word in S and B

$o(c, T) =$

- **Left-Arc** if top(S_c) $\leftarrow$ first(B_c) in T
- **Right-Arc** if top(S_c) $\rightarrow$ first(B_c) in T
- **Reduce** if $\exists w < top(S_c) : w \leftrightarrow$ first(B_c) in T
- **Shift** otherwise

If top of the word in buffer is head and top in stack dependent -> store LA relation
If top in stack is head -> RA relation
If below top of stack, connected to 1st word in buffer

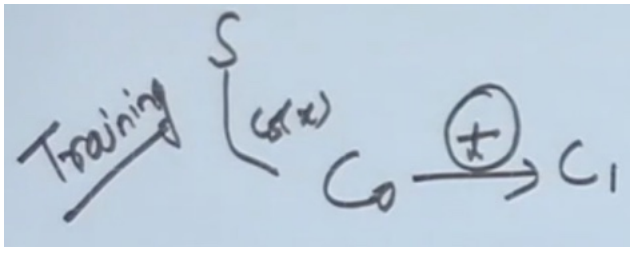
- Otherwise shift

Online Learning with an Oracle

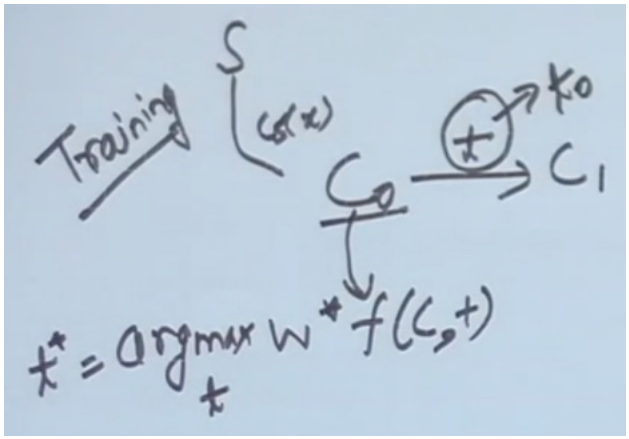
```
LEARN({T_1, ..., T_N})
1    w ← 0.0
2    for i in 1..K
3      for j in 1..N
4        c ← ([]_S, [w_1, ..., w_{n_j}]_B, {})
5        while B_c ≠ []
6          t* ← arg max_t w.f(c, t)
7          t_o ← o(c, T_i)
8          if t* ≠ t_o
9            w ← w + f(c, t_o) − f(c, t*)
10           c ← t_o(c)
11   return w
```

Oracle $o(c, T_i)$ returns the optimal transition of c and T_i

3 Two sets 1..N -> all sentences
4 get init conf
5 while buffer is not empty
6 try to find transition

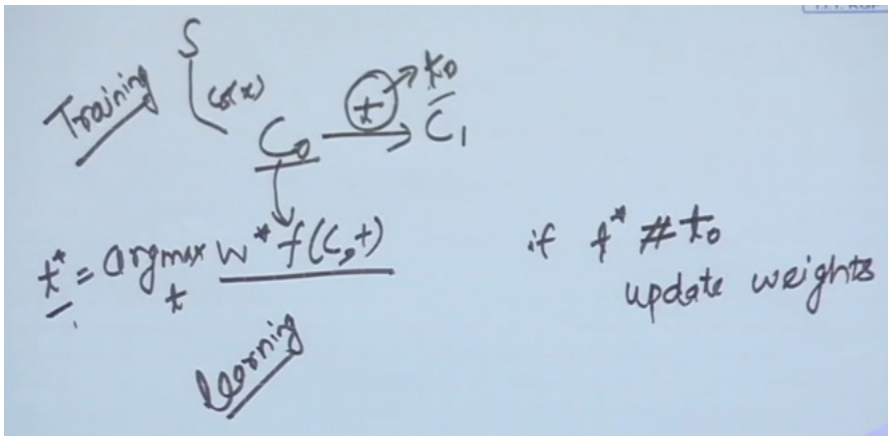


know transition of shifting

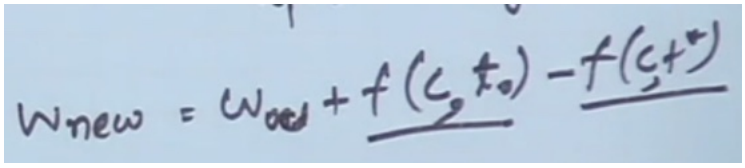


At testing time: convert to feature vector, multiply be weight, take argmax
It is called Optimal transition
Still in learning phase - weight will not be optimal

Adapt weights:

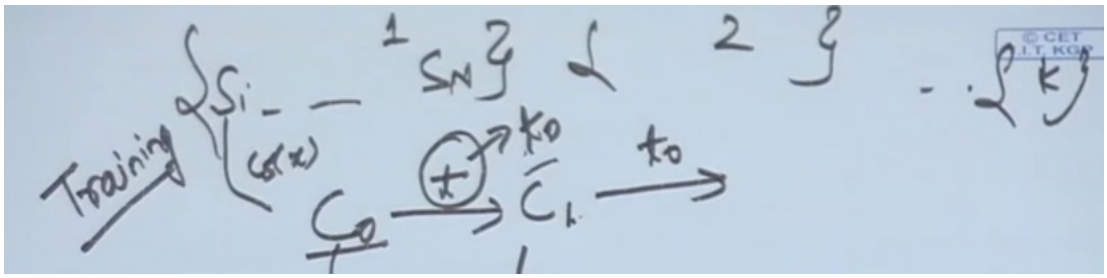


Go in the direction of t0 and away from t*



there will be learning rate

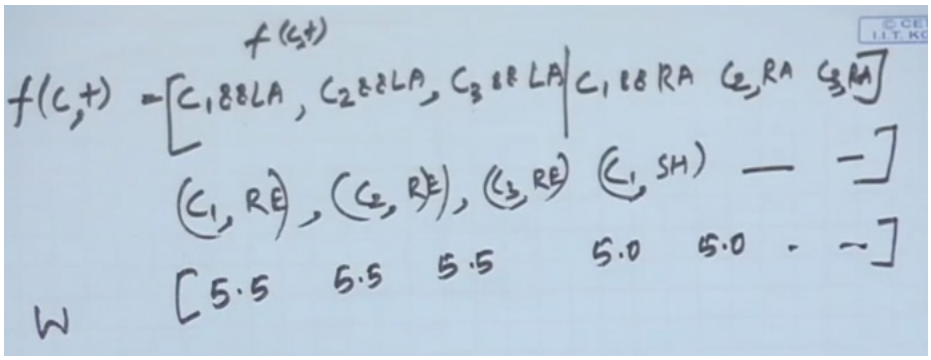
Will have new weight -> do for c1
Do it k times, until the weights are converging:



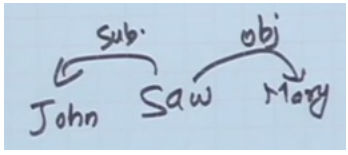
Example

Consider the sentence ‘John saw Mary’

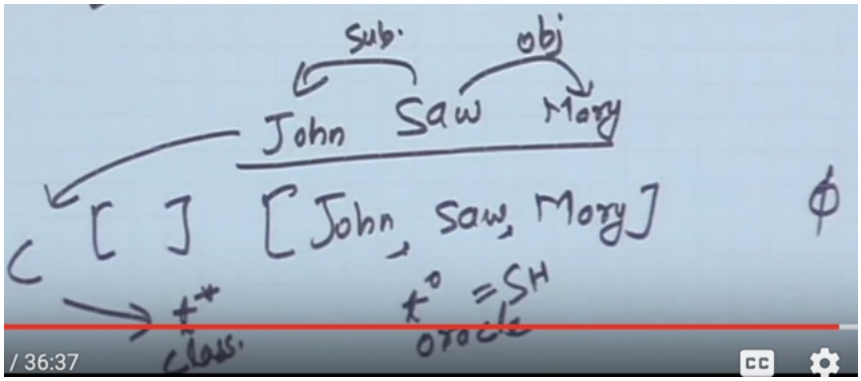
- Draw a dependency graph for this sentence.
- Assume that you are learning a classifier for the data-driven deterministic parsing and the above sentence is a gold-standard parse in your training data. You are also given that John and Mary are ‘Nouns’, while the POS tag of saw is ‘Verb’. Assume that your features correspond to the following conditions:
    - The stack is empty
    - Top of stack is Noun and Top of buffer is Verb
    - Top of stack is Verb and Top of buffer is Noun
- Initialize the weights of all your features to 5.0, except that in all of the above cases, you give a weight of 5.5 to *Left-Arc*. Define your feature vector and the initial weight vector.
- Use this gold standard parse during online learning and report the weights after completing one full iteration of Arc-Eager parsing over this sentence.



Initial weight vectors



- Use this gold standard parse during online learning and report the weights after completing one full iteration of Arc-Eager parsing over this sentence.



What is being predicted by classifier t*

For LA:

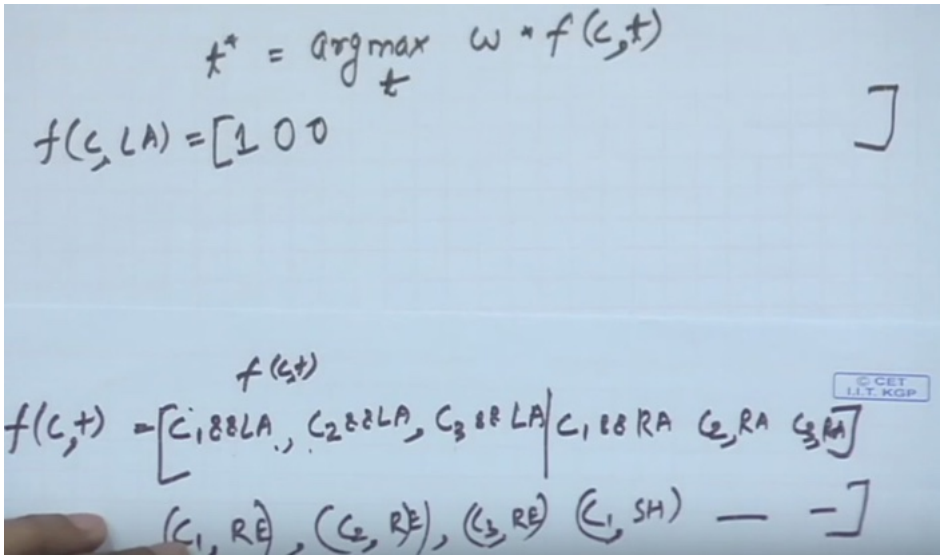
C1 verb

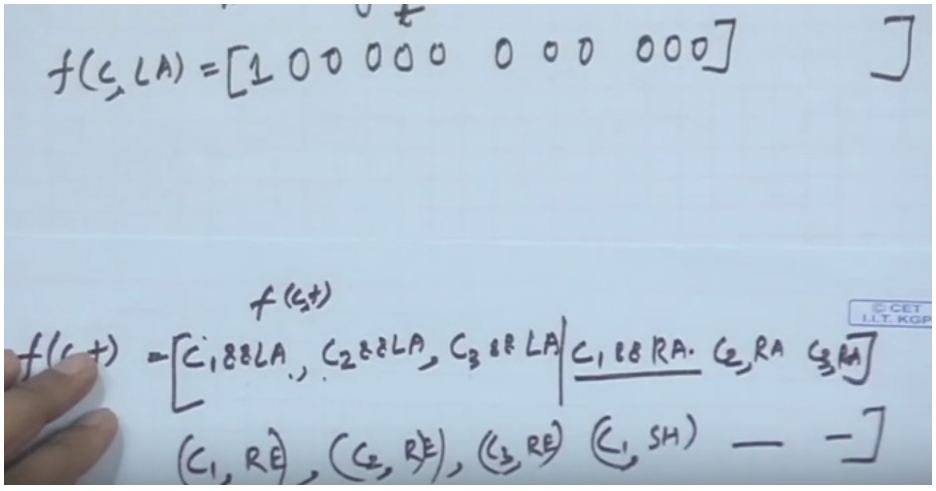
C2 noun - can not contain

C1: The stack is empty

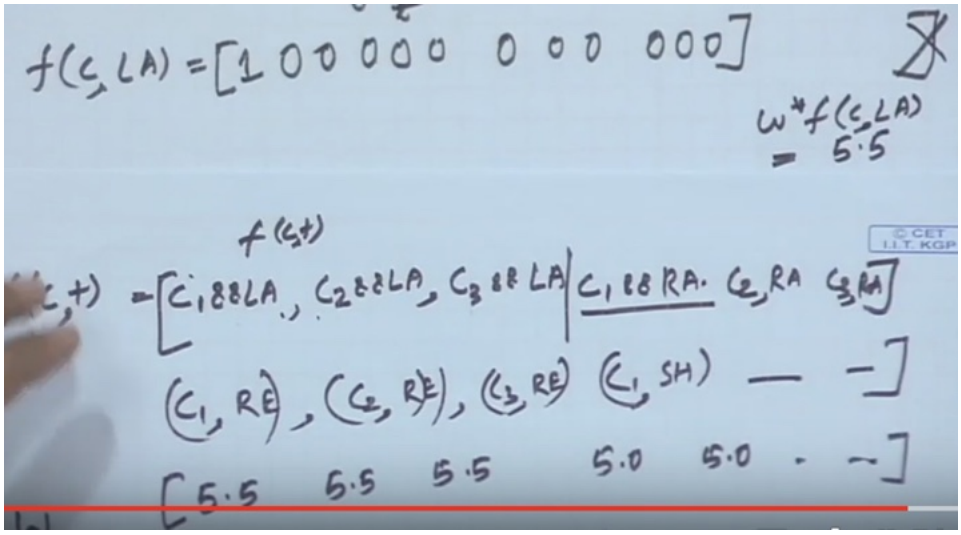
C2: Top of stack is Noun and Top of buffer is Verb

C3: Top of stack is Verb and Top of buffer is Noun



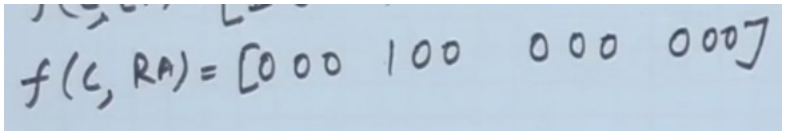


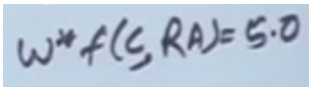
Because of RA everything else will be zero



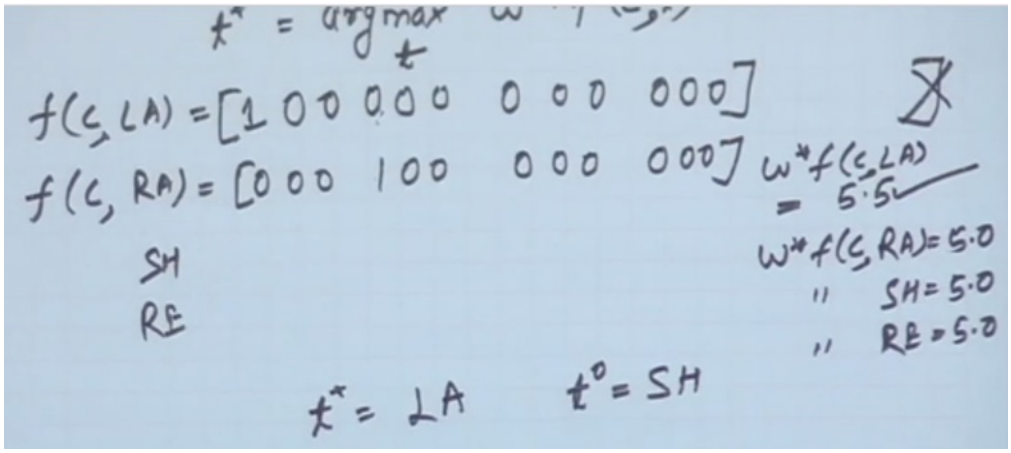
Will get w*f(c,LA) = 5.5 as only one element is 1

Similarly for RA:

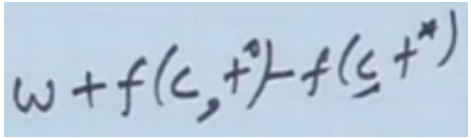




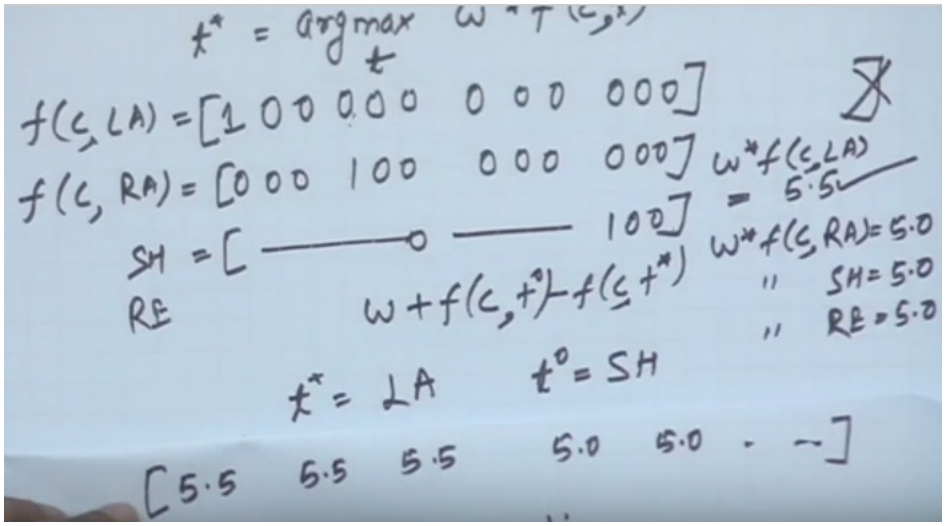
t* will be argmax ->

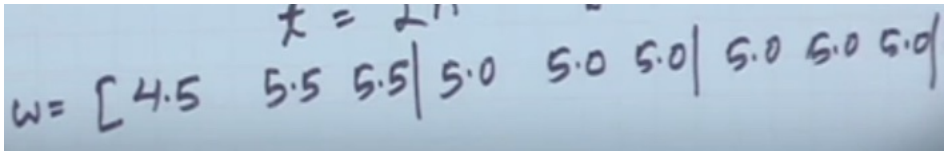


If t* is != t0 update the weights



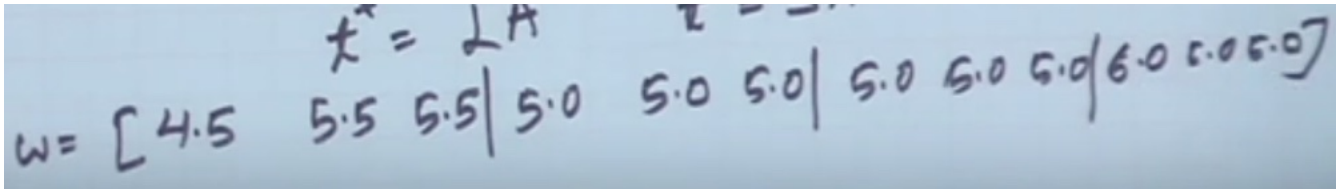
Fc-t0 = fc-SH



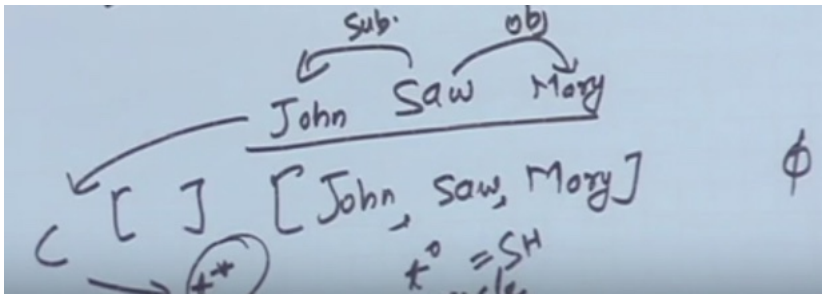


Subtract 1, and zeros

Now come to SH



NEXT computation:



Move John to Stack

- What t* and t0

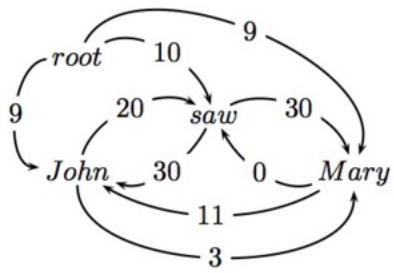
Continue until reach a terminal condition

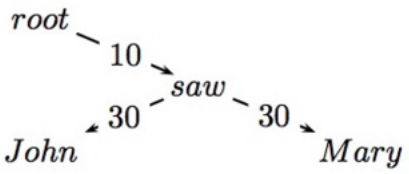
w6L29: W6L4: MST-based Dependency Parsing

Maximum Spanning Tree Based

Basic Idea Starting from all possible connections, find the maximum spanning tree.

-





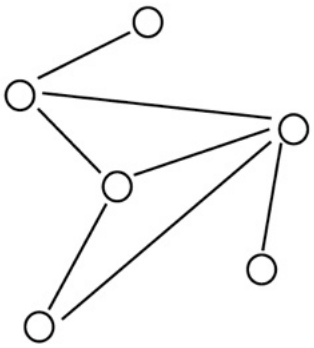
Sentence is set of possible nodes

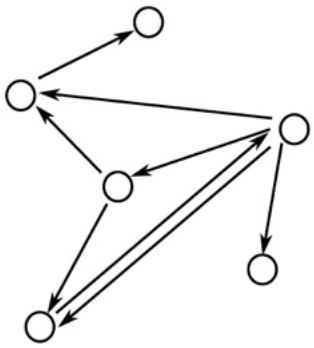
- Assume - root
- Assume - All possible connections
- Find out - what is max spanning tree - this is corresponding to my dependency graph
- Learn edge weight
- What is the max spanning tree -

Some Graph Theory Reminders

V - vertices, A - arches

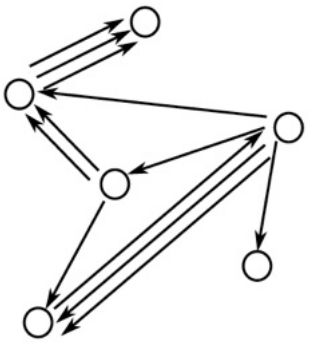
- A graph $G = (V, A)$ is a set of vertices V and arcs $(i, j) \in A$ where $i, j \in V$.
- Undirected graphs: $(i, j) \in A \Leftrightarrow (j, i) \in A$
- Directed graphs (digraphs) : $(i, j) \in A \nRightarrow (j, i) \in A$





Multi-Digraphs

- A multi-digraph is a digraph where multiple arcs between vertices are possible
- $(i,j,k) \in A$ represents the k^{th} arc from vertex i to vertex j.

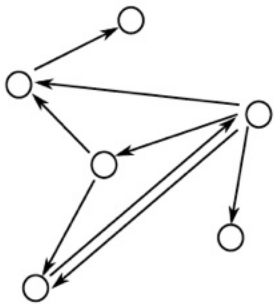


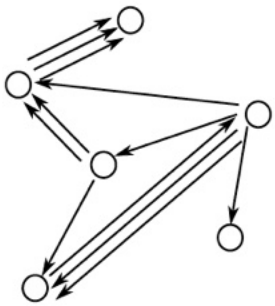
more than one possible arches, additional index i, j, k

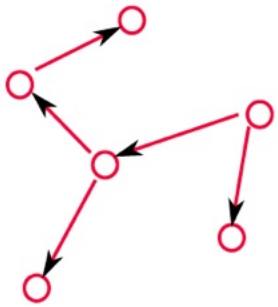
Directed Spanning Trees of a Multi-digraph

- A directed spanning tree of a (multi-)digraph $G = (V,A)$ is a subgraph $G' = (V',A')$ such that:
  - $V' = V$
  - $A' \subseteq A$, and $|A'| = |V'| - 1$
  - G' is a tree (acyclic)
- A spanning tree of the following (multi-)digraphs

1. V' = V Set of nodes are same
2. No. of archs = V-1
3. G' is tree - acyclic







Where there were bidirectional arrows - can still replace it

Weighted Directed Spanning Trees

- Assume we have a weight function for each arc in a multi-digraph $G = (V, A)$.
- Define $w_{ij}^{k} \geq 0$ to be the weight of $(i, j, k) \in A$ for a multi-digraph
- Define the weight of directed spanning tree G' of graph G as

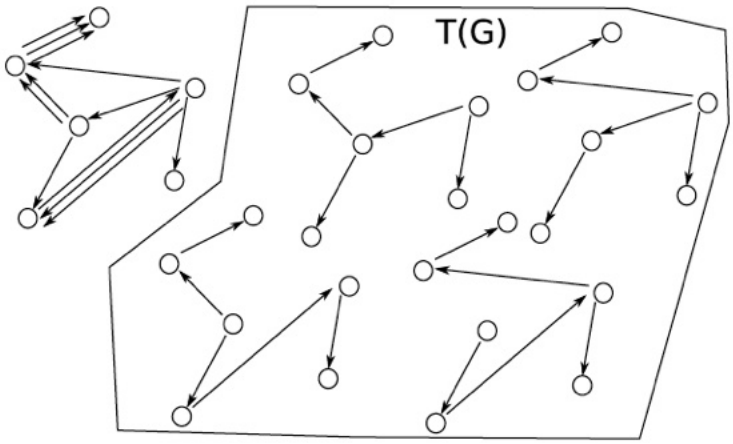
With arch associate weight wij-k

$$w(G') = \sum_{(i,j,k) \in G'} w_{ij}^{k}$$

Which of these - which has maximum weight

Maximum Spanning Trees (MST)

Let T(G) be the set of all spanning trees for graph G



The MST problem

Find the spanning tree G' of the graph G that has the highest weight

$$G' = \arg\max_{G' \in T(G)} w(G') = \arg\max_{G' \in T(G)} \sum_{(i,j,k) \in G'} w_{ij}^{k}$$

From sentence

- Fixed set of Directed trees are possible
- How do i convert sentence to initial configuration
- How do we define weigh
- How do i choose max spanning tree

Finding MST

Directed Graph

For each sentence x, define the directed graph $G_x = (V_x, E_x)$ given by

$$V_x = \{x_0 = root, x_1, \ldots, x_n\}$$

$$E_x = \{(i,j) : i \neq j, (i,j) \in [0:n] \times [1:n]\}$$

Contains addition node - root

- No self edge i != j
- Root will not have incoming [0:n]. Root can have outgoing edge [1:n]

Gx is a graph with

- the sentence **words** and the dummy **root** symbol as **vertices** and
- a directed edge between every pair of **distinct** words and
- a directed edge from the **root** symbol to every word

Can find out the weights of the edges, how do i find MST

Chu-Liu-Edmonds Algorithm

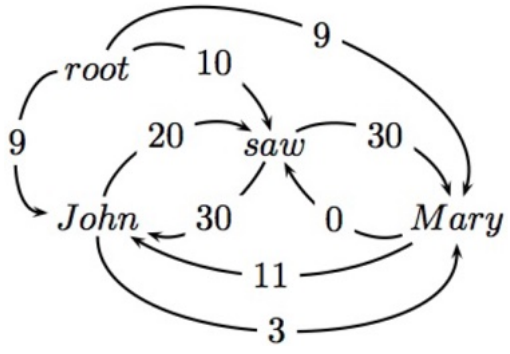
Chu-Liu-Edmonds Algorithm

- Each vertex in the graph **greedily** selects the incoming edge with the **highest** weight.
- If the result graph is tree, it must be a maximum spanning tree.
- If not, there must be a cycle.
    - Identify the cycle and **contract** it into a single vertex.
    - Recalculate edge weights going into and out of the cycle.
- Re-run the algorithm
- You will converge at some step

Chu-Liu-Edmonds Algorithm

x = John saw Mary

- Build the directed graph



how to compute weight for incoming and outgoing arc

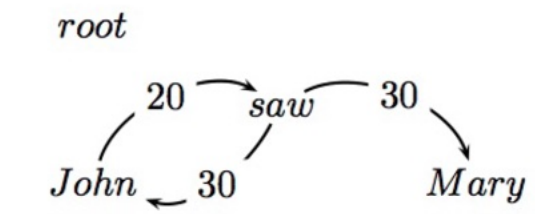
Every word (except root) will have incoming and outgoing edges

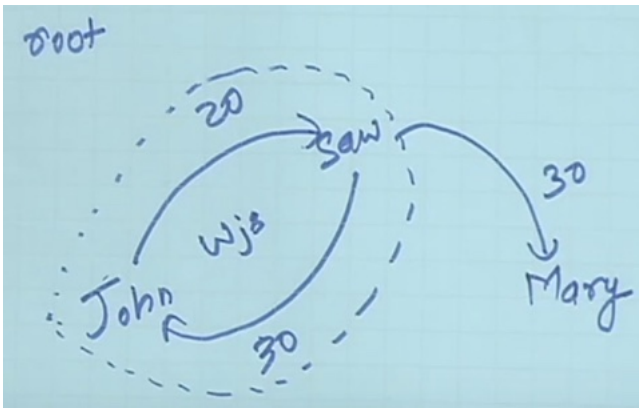
Step 1: Find the highest scoring incoming arc for each vertex
Saw will chose 20
John -30, Mary - 30

Remove all other edges

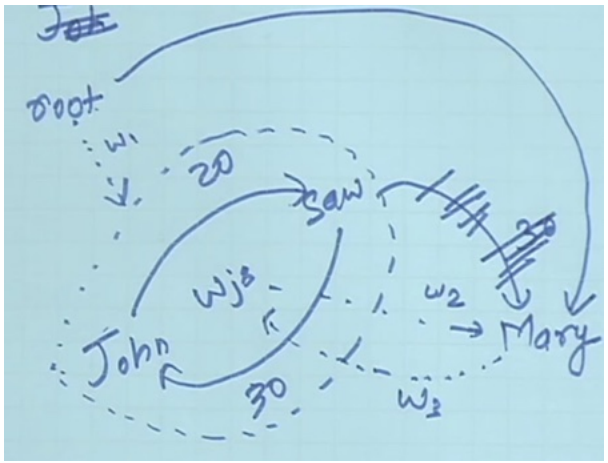
Conditions:
1. No. of nodes same
2. Edges one less than original nodes
3. If this is a tree, then we have found MST.
   Is there a cycle,
   - Yes so contract

New vertex wjs
- What are the incoming and outgoing edges

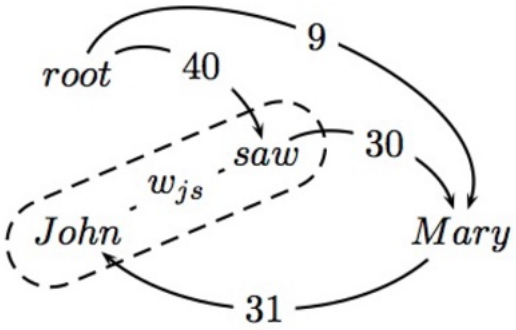


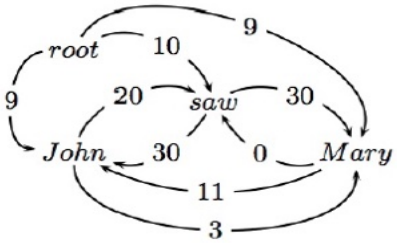


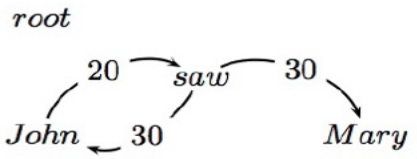
Have to compute w1, w2 and w3

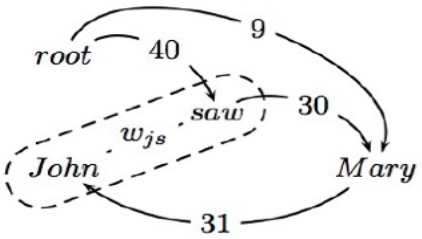


- If not a tree, identify cycle and contract
- Recalculate arc weights into and out-of cycle



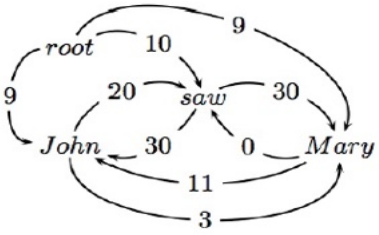


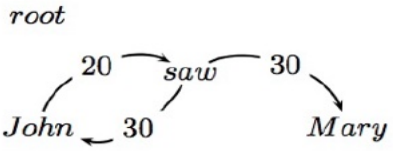


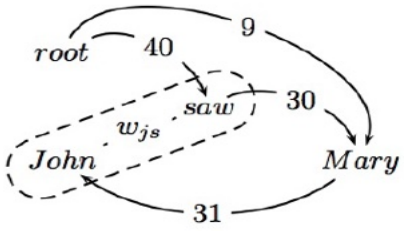


Outgoing arc weights

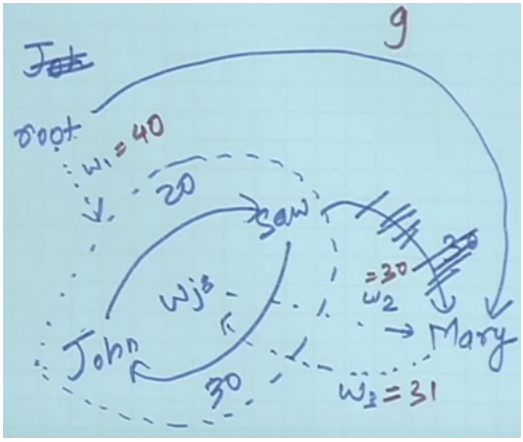
- 1st graph: Equal to the **max** of outgoing arc over all vertices in cycle
- e.g., John -> Mary is 3 and saw -> Mary is 30. Highest is 30
- Put 30 in 3rd graph

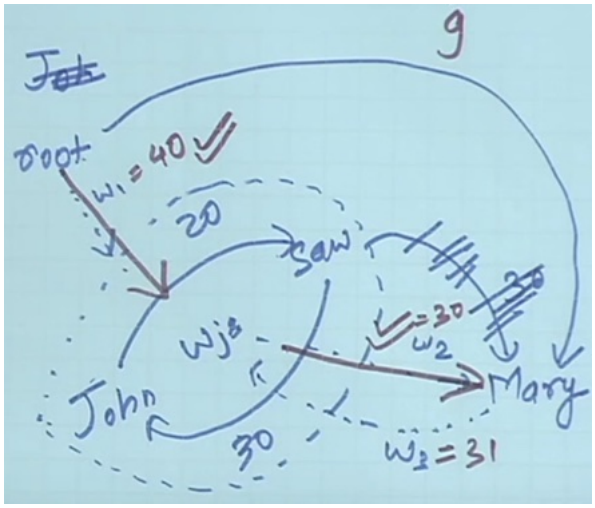






Incoming arc weights

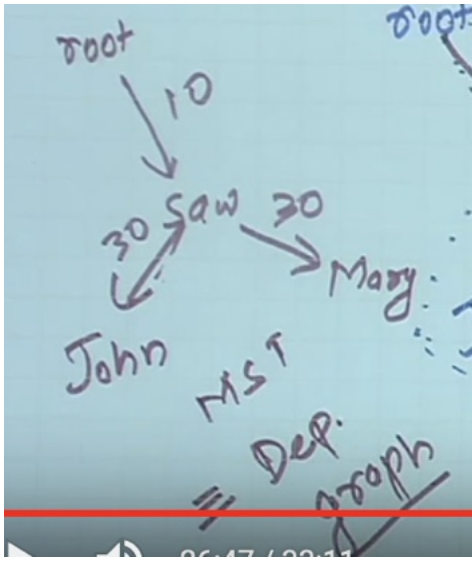
• Equal to the weight of **best spanning tree** that includes head of incoming arc and all nodes in cycle • root -> saw -> John is 40 (10+30) • root -> John -> saw is 29 (9 + 20) • Take weight from Root to Saw as 40 • Mary to John -> 31	

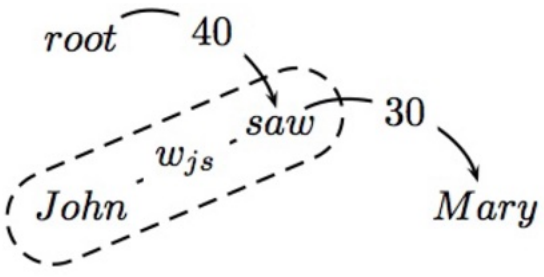


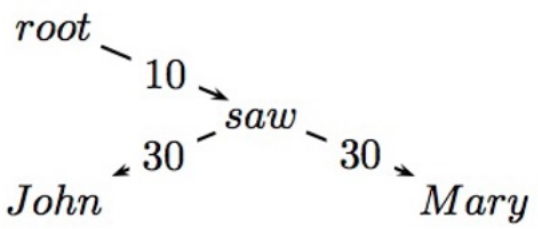
it is a tree -> It is MST

Still haven't obtained Dependency graph for Contracted word

Outgoin edge to Mary was from Saw
Incoming root to saw and saw to john



• Calling the algorithm again on the contracted graph: • This is a tree and the MST for the contracted graph • Go back up the recursive call and reconstruct final graph	



- The edge from w_{js} to *Mary* was from *saw*
- The edge from *root* to w_{js} represented a tree from *root* to *saw* to *John*.

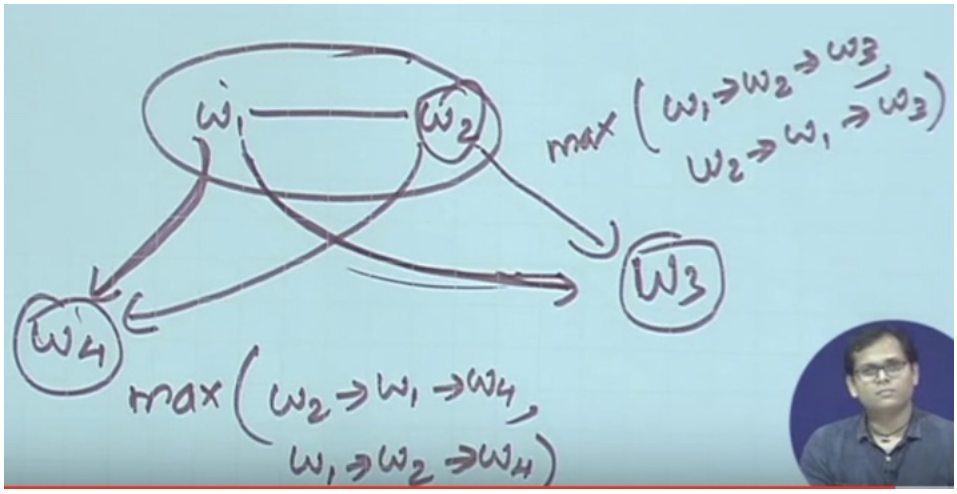
Question:
Different ways of computing incoming and outgoing

- Can I say outing weight should be max

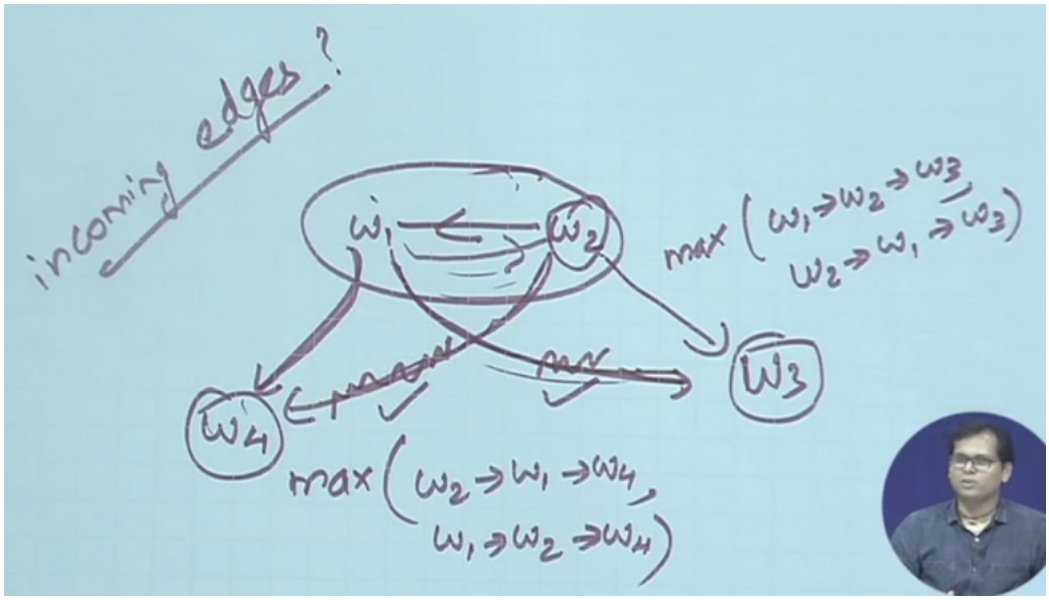
Outgoing is max of w1, w2, another word w4

Each vertex will chose only one arch

- Whatever i obtain - is valid
- From single node -> you can have two outgoing



In final tree you find out other two are max


incoming edges?
w1
w2
w3
w4
max (w1→w2→w3, w2→w1→w3)
max (w2→w1→w4, w1→w2→w4)

w6L30: W6L5: MST-based Dependency Parsing: Learning

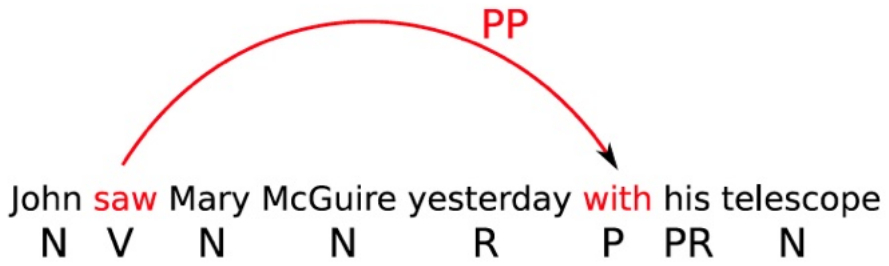
How do we find the edge weights?

Arc weights as linear classifiers

$$w_{ij}{}^{k} = w.f(i,j,k)$$

- Arc weights are a linear combination of features of the arc *f(i, j, k)* and a corresponding weight vector w
- What arc features?

Arc Features f(i, j, k)



Features

Identities of the words w_i and w_j for a label l_k
head = saw & dependent=with

Part-of-speech tags of the words w_i and w_j for a label l_k
head-pos = Verb & dependent-pos=Preposition

Part-of-speech of words surrounding and between w_i and w_j

inbetween-pos = Noun
inbetween-pos = Adverb
dependent-pos-right = Pronoun
head-pos-left=Noun

Number of words between w_i and w_j, and their orientation

arc-distance = 3
arc-direction = right

- Combinations
  head-pos=Verb & dependent-pos=Preposition & arc-label=PP
  head-pos=Verb & dependent=with & arc-distance=3
- No limit : any feature over arc (i,j,k) or input x

Learning the parameters

• Rewrite the inference problem • Which can be plugged into learning algorithms	$G = \arg\max_{G \in T(G_x)} \sum_{(i,j,k) \in G} w_{ij}^{k}$ $= \arg\max_{G \in T(G_x)} w \cdot \sum_{(i,j,k) \in G} f(i,j,k)$ $= \arg\max_{G \in T(G_x)} w \cdot f(G)$

Inference-based Learning

Training data: $T = \{(x_t, G_t)\}_{t=1}^{|T|}$

1. $w^{(0)} = 0; i = 0$
2. for $n : 1..N$
3. for $t : 1..|T|$
4. Let $G' = argmax_{G'} w^{(i)} . f(G')$
5. if $G' \neq G_t$
6. $w^{(i+1)} = w^{(i)} + f(G_t) - f(G')$
7. $i = i + 1$
8. return w^i

Example

Suppose you are training MST Parser for dependency and the sentence, "John saw Mary" occurs in the training set. Also, for simplicity, assume that there is only one dependency relation, "rel". Thus, for every arc from word w_i to w_j, your features may be simplified to depend only on words w_i and w_j and not on the relation label.

Below is the set of features

- f_1: pos(w_i) = Noun and pos(w_j) = Noun
- f_2: pos(w_i) = Verb and pos(w_j) = Noun
- f_3: w_i = Root and pos(w_j) = Verb
- f_4: w_i = Root and pos(w_j) = Noun
- f_5: w_i = Root and w_j occurs at the end of sentence
- f_6: w_i occurs before w_j in the sentence
- f_7: pos(w_i) = Noun and pos(w_j) = Verb

The feature weights before the start of the iteration are: $\{3, 20, 15, 12, 1, 10, 20\}$. Determine the weights after an iteration over this example.